

universidad de león Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

# Ingeniería del software seguro en la era de la IA generativa

## Diseño y Programación Seguras

---

## Verificación de código generado

Francisco J. Rodríguez Lera






# Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** verificación de validación y los cinco tipos de evidencia.
- **Especificar** contratos e invariantes como precondición de la verificación.
- **Detectar** los modos de fallo de los contratos en código generado.
- **Diseñar** pruebas negativas y verificación composicional.
- **Calibrar** la verificación según el riesgo y documentar la aceptación.

# El problema de la confianza delegada

- La necesidad de verificar no depende de quién escribió el código: humano, biblioteca de terceros y fragmento generado pueden incumplir los mismos requisitos.
- La procedencia del artefacto *no* es prueba de corrección.
- Sí influye en la disponibilidad de requisitos, la trazabilidad de decisiones y la independencia de los mecanismos que generan y comprueban el código.

Pregunta operativa: ¿qué propiedades deben cumplirse, con qué técnicas pueden comprobarse y qué limitaciones tiene cada comprobación?

# Qué significa verificar

- Verificar = obtener evidencia de que una implementación satisface propiedades *previamente establecidas*.
- **Verificación formal:** demuestra propiedades bajo hipótesis explícitas.
- **Evaluación mediante pruebas:** observa ejecuciones seleccionadas; no establece por sí sola corrección para todas las entradas.
- Análisis estático, revisión y ejecución instrumentada aportan evidencias complementarias, no equivalentes.
- **Corrección parcial** (si termina, se cumple la poscondición) frente a **total** (además termina).

# Alcance y límites de los mecanismos de verificación

| Mecanismo | Evidencia que aporta | Límite principal |
|---|---|---|
| Compilación | Conformidad con las reglas del compilador | No prueba corrección funcional |
| Pruebas de ejemplo | Comportamiento en ejecuciones seleccionadas | No cubren entradas no ejercitadas |
| Pruebas de propiedades | Comportamiento sobre entradas generadas | Dependen del oráculo y la estrategia |
| Análisis estático | Hallazgos o garantías en su modelo | Depende de supuestos y precisión |
| Revisión humana/asistida | Defectos detectados por inspección | No es demostración exhaustiva |
| Verificación formal | Propiedad demostrada bajo hipótesis | No garantiza que la especificación sea la deseada |

# Cinco tipos de evidencia

1. **Ejecución satisfactoria:** se ejecutó sin observar fallos; respalda los casos ejercitados.
2. **Cumplimiento de propiedades:** una propiedad enunciada sobre entradas generadas.
3. **Garantía formal:** demostrada bajo hipótesis; para todo el dominio del modelo.
4. **Comportamiento en integración:** compatibilidad entre componentes o con el sistema.
5. **Comportamiento en operación:** observado en ejecución real (capítulo de metodología).

Una afirmación de corrección debe indicar *qué tipo* de evidencia la respalda. Confundir ejecución satisfactoria con garantía formal es el error que el capítulo previene.

# Verificación y validación

| Dimensión | Desarrollo humano | Desarrollo asistido por IA |
|---|---|---|
| Requisitos | Pueden quedar implícitos en el autor | Dispersos entre prompt, contexto y documentos |
| Trazabilidad | Consultar al autor (no es prueba) | Las explicaciones generadas no acreditan por sí solas |
| Defectos | Requisitos, diseño, implementación | Los mismos + riesgos de contexto y herramientas |
| Pruebas | Pueden compartir sesgos con el autor | Pueden compartir supuestos con generador y revisor |
| Aceptación | Requiere evidencias técnicas | Requiere las mismas evidencias técnicas |

La tabla describe *posibilidades*, no diferencias universales ni tasas de fallo medidas. La validación responde a «¿satisface la necesidad real de uso?».

# Especificación como precondición de la verificación

- No se puede demostrar conformidad con requisitos que no se han formulado.
- Se pide al modelo «genera una función que valide entradas» sin definir qué es válido, qué ocurre en los límites ni qué propiedad preservar; el defecto queda latente.
- La especificación no necesita ser formal: basta enunciar precondiciones, poscondiciones e invariantes.
- La diferencia entre útil e inútil no es el formalismo, sino la **falsabilidad**: permite diseñar pruebas que la contradigan.

«Manejar entradas grandes» no es falsable; «rechazar cadenas de más de 4096 bytes con error explícito y sin operaciones costosas» sí lo es.

# Formalización mínima y comparación entre autores

- Tripleta de Hoare $\{P\}\ C\ \{Q\}$: si se cumple $P$ y $C$ termina, se cumple $Q$; la terminación requiere una obligación adicional.
- En una escritura: $P$ exige ruta autorizada y entrada acotada; $Q$ exige archivo válido o fallo documentado sin salida parcial.
- En código humano, el contrato se reconstruye por requisitos, revisión y consulta al autor.
- En código generado, el *prompt* no sustituye una especificación verificable; pedir al mismo modelo el contrato puede reproducir sus omisiones.
- En ambos casos, el contrato debe contrastarse con requisitos *externos* al código.

# Contratos en código generado: dónde fallan

**Precondiciones implícitas.** Asume entradas bien formadas sin verificarlo (analizador de CSV que asume comillas correctas y traslada la validación al llamante sin documentarlo).

**Poscondiciones parciales.** Garantiza el caso de éxito pero no el estado tras un fallo: ¿se libera el recurso? ¿queda el archivo consistente?

**Invariantes de seguridad omitidas.** El generador no recibió la invariante y no la infiere (destino dentro del directorio autorizado).

**Supuestos de llamada única.** Asume un único hilo; una comprobación de ruta seguida de apertura puede ser correcta en secuencial e incorrecta en concurrente.

# Contrato implícito (lo que suele generar un modelo)

```python
# Sin contrato explicito (lo que suele generar un modelo sin instrucciones)
def escribir_xlsx(registros, ruta_salida):
    libro = openpyxl.Workbook(); hoja = libro.active
    for fila in registros:
        hoja.append([escapar_celda(c) for c in fila])
    libro.save(ruta_salida)   # Sin verificar el directorio autorizado
```

Listing 1: Sin contrato explícito: funciona para entradas válidas y no ofrece garantías en otro caso.

# Contrato explícito (ilustrativo, NO apto para producción)

```python
# Pre: registros <= MAX_FILAS; ruta dentro de DIRECTORIO_SALIDA
# Post: XLSX con celdas escapadas, o error sin archivo parcial
# Inv:  no se escribe fuera de DIRECTORIO_SALIDA bajo ninguna ruta
def escribir_xlsx_seguro(registros, ruta_salida, directorio_salida):
    ruta = pathlib.Path(ruta_salida).resolve()
    base = pathlib.Path(directorio_salida).resolve()
    if not str(ruta).startswith(str(base) + os.sep):
        raise ValueError(f"Ruta fuera del directorio autorizado: {ruta}")
    if len(registros) > MAX_FILAS:
        raise ValueError(f"Demasiados registros: {len(registros)}")
    tmp = ruta.with_suffix(".tmp")
    try:
        libro = openpyxl.Workbook(); hoja = libro.active
        for fila in registros:
            hoja.append([escapar_celda(str(c)) for c in fila])
        libro.save(tmp); tmp.replace(ruta)
    except Exception:
        tmp.unlink(missing_ok=True); raise
```

Listing 2: Ilustra pre/postcondiciones e invariante; no demuestra contención ante concurrencia ni symlinks.

# El ejemplo es deliberadamente incompleto

- La segunda versión documenta y comprueba algunas condiciones, pero *no demuestra* por sí sola la poscondición ni la contención ante concurrencia o enlaces simbólicos.
- La comprobación previa de rutas puede sufrir condiciones de carrera (*TOCTOU*).
- Una ruta temporal predecible puede ser manipulada.
- El renombrado no garantiza por sí solo persistencia ni atomicidad en todos los entornos.
- Debe comprobarse la extensión requerida por la biblioteca XLSX.

Una implementación robusta necesita operaciones de FS adecuadas al SO, manejo seguro de temporales y pruebas de fallos. El listado *no* debe reutilizarse en producción.

# Invariantes como propiedades verificables

- Una invariante es una propiedad que debe mantenerse en los estados o puntos definidos por su alcance.
- Puede ser de bucle, de representación, de estado de un componente o de seguridad; no todas son globales.
- Las **invariantes de seguridad** son las que, al violarse, tienen consecuencias directas.

# Tres invariantes del caso transversal

1. **Contención de escritura:** ninguna operación escribe fuera del directorio autorizado, ante `../`, rutas absolutas o enlaces simbólicos.
2. **Ausencia de fórmulas:** ninguna celda XLSX contiene contenido interpretable como fórmula, aunque el CSV tenga campos que empiecen por =, +, −, @.
3. **Acotación de recursos:** el proceso termina en tiempo y memoria acotados, incluso con archivos máximos o diseñados para expandirse (p. ej. ZIP con ratio extremo).

Para ser verificables deben enunciarse con precisión suficiente para diseñar pruebas que las contradigan. Las invariantes *no* se obtienen pidiendo «genera código seguro»: deben especificarse, comunicarse al generador y verificarse independientemente.

# Modelo de amenazas como instrumento de especificación

**Activos.** Qué se protege: datos de usuario, disponibilidad, integridad del FS, confidencialidad de credenciales.

**Actores.** Quién puede comprometerlos: usuarios sin autenticar, autenticados con privilegios limitados, atacantes con acceso a red, procesos internos comprometidos.

**Vectores.** Cómo: entradas no validadas, archivos cargados, parámetros de configuración, dependencias de terceros, código generado no revisado.

**Supuestos de confianza.** Qué se da por seguro: SO anfitrión, biblioteca estándar, dependencias externas, modelo de IA.

**Controles existentes.** Qué mitiga ya y con qué limitaciones.

# Modelo de amenazas del conversor CSV-PDF-XLSX

| Activo | Vector | Amenaza | Control |
|---|---|---|---|
| Integridad del FS | Nombre de salida derivado de la entrada | *Path traversal* | Canonicalización y restricción (inv. 1) |
| Integridad del XLSX | Contenido de celdas del CSV | Inyección de fórmulas (CWE-1236) | Escapado de prefijos (inv. 2) |
| Disponibilidad | CSV de tamaño extremo/malformado | Consumo excesivo | Límites de tamaño y tiempo (inv. 3) |
| Cadena de suministro | Dependencias de terceros | Vulnerabilidades en bibliotecas | Gestión de dependencias; YAGNI |
| Código generado | Modelo de IA como fuente | Supuestos tácitos; controles omitidos | Especificación; pruebas negativas; revisión |

Debe separarse el modelo del *producto* del del *proceso de generación* (contaminación de contexto, instrucciones maliciosas, filtración de secretos). El generador no necesita ser adversario intencional: es un componente no confiable o superficie de ataque.

# Pruebas negativas y propiedades

- Las pruebas **positivas** verifican que hace lo correcto con entradas válidas.
- Las **negativas** verifican que *no* hace lo indebido con entradas inválidas, malformadas o maliciosas.
- Un generador al que se pide «escribe pruebas» tiende a producir casos normales; los anómalos exigen especificar las invariantes y las clases de entradas que las violan.
- Las rutas de error y los valores límite son los que los atacantes explotan: manejar el 99,9 % correctamente no protege del 0,1 % no probado.

# Pruebas positivas y negativas (contención de escritura)

```python
# --- Pruebas positivas ---
def test_escritura_valida(registros_simples):
    ruta = DIRECTORIO_SALIDA / "resultado.xlsx"
    escribir_xlsx_seguro(registros_simples, ruta, DIRECTORIO_SALIDA)
    assert ruta.exists()

# --- Negativas: contencion de escritura ---
@pytest.mark.parametrize("ruta_mala", [
    "/tmp/fuera.xlsx",                  # Ruta absoluta fuera
    "../fuera.xlsx",                    # Path traversal relativo
    "/tmp/salida_test/../fuera.xlsx", # Traversal con componente valida
])
def test_path_traversal_rechazado(ruta_mala, registros_simples):
    with pytest.raises(ValueError, match="fuera del directorio autorizado"):
        escribir_xlsx_seguro(registros_simples, ruta_mala, DIRECTORIO_SALIDA)
```

Listing 3: Ejemplo didáctico: pruebas vinculadas a las invariantes enunciadas.

# Prueba negativa: ausencia de fórmulas

```python
# --- Negativas: ausencia de formulas ---
@pytest.mark.parametrize("prefijo", ["=SUMA(A1)", "+CMD", "-1+1", "@SUM(1,1)"])
def test_formula_escapada(prefijo):
    ruta = DIRECTORIO_SALIDA / "formulas.xlsx"
    escribir_xlsx_seguro([[prefijo, "valor"]], ruta, DIRECTORIO_SALIDA)
    celda = openpyxl.load_workbook(ruta).active.cell(row=1, column=1).value
    assert not any(str(celda).startswith(p) for p in ("=", "+", "-", "@"))
```

Listing 4: El oráculo comprueba el tipo/contenido de la celda, no solo que el archivo existe.

# Selección de técnicas y comparación entre autores

- Particiones de equivalencia y valores límite derivan casos del dominio.
- Pruebas basadas en propiedades contrastan invariantes sobre múltiples entradas.
- Metamórficas: relaciones entre ejecuciones; *fuzzing*: entradas inesperadas.
- Mutación: estiman si la batería detecta cambios artificiales.
- Cada instrumento necesita oráculo e interpretación de sus límites.
- Un modelo que genera código y pruebas del mismo contexto puede repetir las mismas omisiones; una batería derivada de requisitos y amenazas reduce esa dependencia, sin garantizar independencia completa.

Las pruebas negativas no demuestran seguridad; demuestran que las propiedades especificadas se preservan en los casos probados.

# Verificación independiente en capas

1. **Especificación antes de la generación:** contrato y modelo de amenazas; alimenta `CLAUDE.md`/`REVIEW.md`.
2. **Verificación interna por el agente:** pruebas, compilador y estático; entrega también la salida de las herramientas.
3. **Revisión automática independiente:** agente revisor con contexto fresco sobre el diff.
4. **Revisión de seguridad especializada:** capa separada que bloquea la integración.
5. **Revisión humana calibrada por riesgo:** arquitectura, alta consecuencia, especificación ambigua.
6. **Integración y detección rápida:** verificación adicional en la rama principal; rollback.

El artículo de referencia [1] documenta que, entre junio y septiembre de 2026, los PR integrados más que se duplicaron mientras la tasa de reversiones cayó más de un 40 %: evidencia contextual, no generalizable sin adaptación.

# La convergencia del ciclo de revisión

- El mismo artículo documenta un caso de un agente autor y otro revisor con más de cincuenta rondas sin convergencia, por detectar problemas en código preexistente no modificado.
- Solución: limitar rondas, restringir a cambios incrementales, prohibir repetir hallazgos.
- Implicación general: la verificación automática debe diseñarse para *converger*.
- Un proceso que no converge no es una propiedad de seguridad: es ruido que degrada la señal.

# Caso de estudio: humano frente a IA (protocolo)

1. **Congelar requisitos:** formatos, rutas, límites, política de errores, fórmulas.
2. **Obtener implementaciones** (A y B), registrando dependencias, instrucciones, tiempo y herramientas.
3. **Oráculo independiente:** derivar contratos, invariantes y pruebas de los requisitos, sin ver el código.
4. **Misma batería:** compilación/análisis, pruebas nominales, límites, malformadas, propiedades, amenazas.
5. **Registrar evidencias:** comandos, resultados, versiones, cobertura, defectos reproducibles.

No se presupone cuál será más correcta; un caso didáctico no autoriza conclusiones estadísticas sin replicación, controles y análisis.

# Evidencias de aceptación y trazabilidad

- La aceptación requiere un expediente mínimo: versión del código, requisitos, contratos, invariantes, amenazas, resultados de herramientas, incidencias abiertas y decisión sobre riesgos residuales.
- Debe poder revisarse *sin* depender de afirmaciones del autor, sea persona o modelo.
- Para cada propiedad crítica: (1) formulación comprobable; (2) mecanismo y resultado; (3) supuestos y límites; (4) decisión explícita sobre lo no verificado.

# Verificar no es aprobar: cinco decisiones

1. **Resultado de la verificación**: si se satisface cada propiedad, con qué evidencia.
2. **Cumplimiento de criterios de aceptación**: si el conjunto de evidencias satisface los criterios.
3. **Autorización de integración**: si puede incorporarse al sistema.
4. **Autorización de despliegue**: si puede exponerse a usuarios reales y con qué mecanismos.
5. **Aceptación del riesgo residual**: si los riesgos no cubiertos son aceptables.

Pueden recaer en la misma persona pero *no son la misma decisión*. Presentar un expediente verde como autorización automática de despliegue confunde la evidencia con la decisión.

# Verificación composicional y contratos entre componentes

- Una función puede satisfacer sus poscondiciones y, aun así, participar incorrectamente en un sistema.
- Contrato *assume-guarantee*: $C_i$ garantiza $G_i$ si su entorno cumple $A_i$.
- Para conectar dos componentes hay que justificar que la salida del primero satisface la precondición del segundo; verificar cada uno aisladamente no lo establece.
- Ejemplo: `Decimal` con seis decimales persistido en una columna de tres; o una enumeración entera que rompe un consumidor que espera cadena.

# Obligaciones de verificación por nivel de composición

| Nivel | Obligación | Evidencia apropiada |
|---|---|---|
| Función | Pre/poscondiciones y terminación | Pruebas de propiedades, análisis o demostración |
| Interfaz | Tipos, unidades, precisión, errores | Pruebas de contrato y validación en límites |
| Integración | Compatibilidad de dependencias, estado, transacciones | Pruebas de integración y escenarios extremos |
| Sistema | Requisitos de extremo a extremo y no funcionales | Pruebas de sistema, monitorización, riesgos |

En flujos con agentes que modifican varios componentes conviene comprobar explícitamente la consistencia entre archivos, esquemas, configuraciones y consumidores.

# Validez de las pruebas y de sus oráculos

- Una batería superada solo es evidencia si ejecuta las rutas relevantes y sus aserciones distinguen resultados correctos de incorrectos.
- El *oráculo* es el criterio que decide si una ejecución cumple la propiedad: resultado esperado, modelo de referencia, relación metamórfica o especificación formal.
- Antes de aceptar una prueba: (i) alcanza el comportamiento; (ii) sus aserciones se ejecutan; (iii) los dobles de prueba no sustituyen lo que se quiere comprobar; (iv) falla ante un defecto representativo.
- Cobertura alta *no* implica aserciones eficaces.

# Defectos de la infraestructura de verificación

| Defecto | Consecuencia | Control |
|---|---|---|
| Aserción omitida o no ejecutada | Falso resultado satisfactorio | Comprobar ejecución y fallos deliberados |
| Datos de prueba incompletos | Ruta relevante nunca alcanzada | Instrumentación y cobertura de ramas |
| Oráculo derivado del resultado observado | Se consolida el defecto como expectativa | Derivar expectativas de requisitos independientes |
| Prueba debilitada o eliminada | Pérdida de detección | Revisar diferencias en pruebas y aserciones |

La IA puede generar código y pruebas del mismo contexto: riesgo de errores correlacionados. En propiedades críticas, derivar oráculos de requisitos independientes y revisar los cambios que suprimen pruebas o introducen *mocks*.

# Seguridad del proceso de generación

- El modelo de amenazas cubre dos objetos: el producto entregado y el entorno que lo genera.
- Un agente que lee incidencias, ejecuta comandos, instala dependencias y modifica archivos tiene superficie de actuación mayor.
- El contenido de repositorios, PR, documentación y respuestas de herramientas es entrada potencialmente no fiable [3, 2].
- Medidas: ejecución aislada, privilegios mínimos, credenciales efímeras, revisión de nuevas dependencias, restricciones de red y FS.
- Inspeccionar *todos* los archivos modificados: pruebas, ficheros de bloqueo, scripts de compilación, configuración de CI. La descripción generada de una PR *no* sustituye el *diff* real.

# Niveles de riesgo y aprobación exigida

| Nivel | Criterios orientativos | Aprobación | Siempre persona |
|---|---|---|---|
| Bajo | Documentación, copias, cambios tras *flag* | Controles automáticos y puertas verdes | - |
| Medio | Cambios ordinarios; áreas sensibles (caché, config.) | Una persona distinta del generador | - |
| Alto | Lógica crítica; cambios amplios; alta exposición | Una persona con revisión de seguridad | - |
| Crítico | Autenticación, autorización, persistencia, secretos, CI/CD | Dos o más; seguridad obligatoria | Sí, siempre |

**Propuesta docente.** El nivel lo determina *el cambio*, no el autor; un cambio pequeño en ruta crítica no es de bajo riesgo. La aprobación automatizada solo es razonable con riesgo bajo, puertas verdes y recuperación efectiva; nunca sustituye una aprobación humana exigida.

## Síntesis del capítulo

- Los contratos explicitan precondiciones, poscondiciones y fallos; las invariantes expresan propiedades que deben mantenerse; los modelos de amenazas identifican escenarios a resistir.
- Pruebas, análisis y métodos formales dan evidencias de *alcance distinto*; el criterio de aceptación debe explicitar lo comprobado y lo que queda.
- Los instrumentos son comunes al código humano y al generado: la IA cambia velocidad, trazabilidad e independencia, *no* qué constituye una prueba técnica de corrección.
- La comparación entre orígenes debe basarse en requisitos, herramientas y resultados observables, no en la plausibilidad.

Las diferencias propuestas entre flujos son hipótesis de trabajo cuando no se acompañan de datos comparativos. El caso de estudio ofrece un protocolo, no resultados inventados.

# Ejercicios

1. **Contrato:** especifique una función CSV→XLSX con pre/poscondiciones y errores; exprésela con una tripleta de Hoare y aclare si demuestra terminación.
2. **Invariantes:** distinga representación, seguridad y propiedad temporal; asigne mecanismo de comprobación y límites.
3. **Amenazas:** amplíe la tabla con dos amenazas al producto y una al flujo con IA.
4. **Pruebas:** amplíe el listado con valores límite y una propiedad generativa.
5. **Comparación:** aplique el protocolo a dos implementaciones sin inferir superioridad general.
6. **Aceptación:** redacte un expediente breve de verificación.

## DISCUSIÓN ¿Cambia la IA qué constituye una prueba de corrección?

- Si el código lo genera un modelo, ¿basta con que «parezca correcto»?

# Referencias I




Addy Osmani, Mark W. Lunt, and Qing Wang.

How the claude code team does code review.

claude.dev Blog, 15 de octubre de 2026, 2026.

Testimonio del equipo Claude Code; no es evidencia experimental independiente.



OWASP.

Ai agent and mcp security.

OWASP DevSecOps Guideline.

Guía de referencia.



OWASP.

Secure coding with ai cheat sheet.

Guía de referencia. Consultado en noviembre de 2026.

# Uso de IA generativa

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.